

UQFFLearningAssessment_001.cpp

UQFF C++ Source Module

```
=====
===== Header: UQFFLearningAssessment.h Description: C++ Module for UQFF Learning
Assessment — Research Student Edition (v2.0) This is the ninth module in a series of 500+
code files for the Universal Quantum Field Framework (UQFF) simulations. Covers ALL SEVEN
completed astrophysical systems in a unified research-student comparative framework: [0]
SGR 1745-2900 — Magnetar (compact, GC proximity, D(t) burst, 18 terms) [1] SGR 0501+4516
— Magnetar (compact, Perseus arm, D(t) burst, 16 terms)
```

Daniel T. Murphy • UQFF Research Consortium • 2026

Lines: 792 Size: 45,984 bytes

Module Description

```
=====
=====
Header: UQFFLearningAssessment.h
```

Description: C++ Module for UQFF Learning Assessment — Research Student Edition (v2.0)

This is the ninth module in a series of 500+ code files for the Universal Quantum Field Framework (UQFF) simulations. Covers ALL SEVEN completed astrophysical systems in a unified research-student comparative framework:

- [0] SGR 1745-2900 — Magnetar (compact, GC proximity, D(t) burst, 18 terms)
- [1] SGR 0501+4516 — Magnetar (compact, Perseus arm, D(t) burst, 16 terms)
- [2] Sgr A* — SMBH (galactic center, M(t) accretion, QPO, 15 terms)
- [3] Starbirth Tapestry — Young cluster (LMC NGC2014/2020, M(t) + wind, 12 terms)
- [4] Westerlund 2 — Super cluster (Carina, M(t) + OB winds, 12 terms)
- [5] Pillars of Creation— Star-forming pillars (Eagle Nebula M16, E(t) erosion, 12 terms)
- [6] Rings of Relativity— Einstein ring (GAL-CLUS-022058s, H(z), L_t lensing, 12 terms)

Purpose: Research-student comparative framework for UQFF mastery assessment.

- Aggregates canonical parameters from all 7 completed C++ modules (Sessions 63-70)
- Computes g_base (Newtonian G^*M/r^2), Ubi (buoyancy Tier-1), and advancement metrics
- Metrics: diversity (7/7 regimes), dynamic processes (7/10 unique), scale span (~16.2 log-decades in r), coverage (7/7 planned modules complete)
- Advancement score: weighted mean of 4 metrics -> 0-100%
- Educational output: printStudentGuide() (unique physics per system, dynamic catalogue,

scale analysis), printComparativeAnalysis() (sorted g-table across all 7 systems)

- Full UQFF 2.0 Self-Expanding Framework (logging, dynamic_params, exportState, cross_validate<>)

Integration: Designed for inclusion in base program 'ziqn233h.cpp' (not present here).

Instantiate class in main: UQFFLearningAssessment assess;

double adv = assess.compute_advancement();

assess.printStudentGuide();

assess.printComparativeAnalysis(1.0e6 * 3.156e7); // at t = 1 Myr

Key Features:

- Canonical parameters sourced directly from all 7 completed UQFF C++ modules
- 7 physical regimes: magnetar x2, SMBH, LMC cluster, MW super-cluster, MW star-forming pillars, cosmological Einstein ring
- Scale range: r = 20 km (magnetar) to 10 kpc (Einstein ring) = 16.2 log-decades
- Mass range: M = 1.4 M_{sun} (magnetar) to 1e14 M_{sun} (galaxy cluster) = ~20 dex
- compute_advancement(): 4-metric weighted UQFF score (diversity, dynamics, scale, coverage)
- compute_g_base(int): Newtonian $g = G \cdot M / r^2$ for system 0 through 6
- compute_Ubi(int): UQFF buoyancy Tier-1 = 0.5 * g_base
- compute_F_UBii(int, double): UQFF buoyancy Tier-2 at time t
- compute_Ub_i(int, double): UQFF buoyancy Tier-3 (external body) at time t
- printStudentGuide(): educational walkthrough + unique physics catalogue
- printComparativeAnalysis(double t): sorted g-table for all 7 systems

Author: Encoded by Grok (xAI), based on Daniel T. Murphy's UQFF manuscript.

Date: March 16, 2026 (Research Student Edition — replaced October 08, 2025 stub)

Copyright: Daniel T. Murphy, daniel.murphy00@gmail.com

=====

=====

Source Code

```

1  /**
2   * =====
3   * Header: UQFFLearningAssessment.h
4   *
5   * Description: C++ Module for UQFF Learning Assessment – Research Student Edition (v2.0)
6   *              This is the ninth module in a series of 500+ code files for the Universal Quantum
7   *              Field Framework (UQFF) simulations. Covers ALL SEVEN completed astrophysical systems
8   *              in a unified research-student comparative framework:
9   *              [0] SGR 1745-2900      – Magnetar (compact, GC proximity, D(t) burst, 18 terms)
10  *              [1] SGR 0501+4516     – Magnetar (compact, Perseus arm, D(t) burst, 16 terms)

```

```

11 *           [2] Sgr A*           – SMBH (galactic center, M(t) accretion, QP0, 15 terms)
12 *           [3] Starbirth Tapestry – Young cluster (LMC NGC2014/2020, M(t) + wind, 12 terms)
13 *           [4] Westerlund 2     – Super cluster (Carina, M(t) + OB winds, 12 terms)
14 *           [5] Pillars of Creation– Star-forming pillars (Eagle Nebula M16, E(t) erosion, 12 terms)
15 *           [6] Rings of Relativity– Einstein ring (GAL-CLUS-022058s, H(z), L_t lensing, 12 terms)
16 *
17 * Purpose: Research-student comparative framework for UQFF mastery assessment.
18 *           - Aggregates canonical parameters from all 7 completed C++ modules (Sessions 63-70)
19 *           - Computes g_base (Newtonian  $G*M/r^2$ ), Ubi (buoyancy Tier-1), and advancement metrics
20 *           - Metrics: diversity (7/7 regimes), dynamic processes (7/10 unique), scale span
21 *             (~16.2 log-decades in r), coverage (7/7 planned modules complete)
22 *           - Advancement score: weighted mean of 4 metrics -> 0-100%
23 *           - Educational output: printStudentGuide() (unique physics per system, dynamic catalogue
24 *             scale analysis), printComparativeAnalysis() (sorted g-table across all 7 systems)
25 *           - Full UQFF 2.0 Self-Expanding Framework (logging, dynamic_params, exportState,
26 *             cross_validate<&>);
27 *
28 * Integration: Designed for inclusion in base program &#39;ziqn233h.cpp&#39; (not present here).
29 *           Instantiate class in main: UQFFLearningAssessment assess;
30 *           double adv = assess.compute_advancement();
31 *           assess.printStudentGuide();
32 *           assess.printComparativeAnalysis(1.0e6 * 3.156e7); // at t = 1 Myr
33 *
34 * Key Features:
35 *           - Canonical parameters sourced directly from all 7 completed UQFF C++ modules
36 *           - 7 physical regimes: magnetar x2, SMBH, LMC cluster, MW super-cluster,
37 *             MW star-forming pillars, cosmological Einstein ring
38 *           - Scale range: r = 20 km (magnetar) to 10 kpc (Einstein ring) = 16.2 log-decades
39 *           - Mass range: M = 1.4 M_sun (magnetar) to 1e14 M_sun (galaxy cluster) = ~20 dex
40 *           - compute_advancement(): 4-metric weighted UQFF score (diversity, dynamics, scale, coverage)
41 *           - compute_g_base(int): Newtonian  $g = G*M/r^2$  for system 0 through 6
42 *           - compute_Ubi(int): UQFF buoyancy Tier-1 = 0.5 * g_base
43 *           - compute_F_UBii(int, double): UQFF buoyancy Tier-2 at time t
44 *           - compute_Ub_i(int, double): UQFF buoyancy Tier-3 (external body) at time t
45 *           - printStudentGuide(): educational walkthrough + unique physics catalogue
46 *           - printComparativeAnalysis(double t): sorted g-table for all 7 systems
47 *
48 * Author: Encoded by Grok (xAI), based on Daniel T. Murphy&#39;s UQFF manuscript.
49 * Date: March 16, 2026 (Research Student Edition – replaced October 08, 2025 stub)
50 * Copyright: Daniel T. Murphy, daniel.murphy00@gmail.com
51 * =====
52 */
53
54 #ifndef UQFF_LEARNING_ASSESSMENT_H
55 #define UQFF_LEARNING_ASSESSMENT_H
56
57 #include <iostream>;
58 #include <cmath>;
59 #include <iomanip>;
60 #include <string>;
61 #include <limits>;
62 #include <map>;
63 #include <fstream>;
64 #include <array>;
65 #include <algorithm>;

```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

396 // Newtonian g = G*M/r^2 for system idx (0 = SGR1745 ... 6 = RINGS)
397 double compute_g_base(int idx) const {
398     if (idx <= 0 || idx >= NUM_SYSTEMS) {
399         std::cerr <<< "Error: system index " <<< idx
400             <<< "out of range [0,<<< (NUM_SYSTEMS - 1) <<< &
401         return 0.0;
402     }
403     double g = g_base_for(idx);
404     if (logging_enabled)
405         std::cout <<< "[LOG] g_base[<<< idx <<< <<< <<< <<<
406             <<< "<<<]=<<< <<< std::scientific <<< g <<< <<< <<<
407     return g;
408 }
409
410 // log10(r_max / r_min) across all 7 systems
411 double compute_scale_range() const {
412     double r_min = r_sgr1745;    // 20 km – magnetar surface
413     double r_max = r_rings;      // 10 kpc – Einstein radius
414     double span = std::log10(r_max / r_min);
415     if (logging_enabled)
416         std::cout <<< "[LOG] scale_range: r_min=<<< r_min
417             <<< "r_max=<<< r_max <<< "log10=<<< <<<
418     return span;
419 }
420
421 // UQFF buoyancy Tier-1: term_Ubi = 0.5 * g_base (CP3/PAPER_198 static half-gravity)
422 double compute_Ubi(int idx) const {
423     return 0.5 * g_base_for(idx);
424 }
425
426 // UQFF buoyancy Tier-2: term_F_UBii = -beta_i * g * omega_g * (M/r) * U-UA * cos(pi*t)
427 double compute_F_UBii(int idx, double t) const {
428     double g = g_base_for(idx);
429     double Mt = get_M(idx);
430     double ri = get_r(idx);
431     return -beta_i * g * omega_g * (Mt / ri) * U-UA * std::cos(UQFF_LA_PI * t);
432 }
433
434 // UQFF buoyancy Tier-3: term_Ub_i = -beta_i * g * omega_g * (M_ext/r_ext) * U-UA * cos(pi*t)
435 double compute_Ub_i(int idx, double t) const {
436     double g = g_base_for(idx);
437     double M_ext = 0.0, r_ext = 1.0;
438     get_ext(idx, M_ext, r_ext);
439     return -beta_i * g * omega_g * (M_ext / r_ext) * U-UA * std::cos(UQFF_LA_PI * t);
440 }
441
442 // ■■ Comparative analysis output ████████████████████████████████████████████████████
443
444 // Print sorted g-value table for all 7 systems at time t (seconds)
445 void printComparativeAnalysis(double t = 0.0, std::ostream& os = std::cout) const {
446     os <<< "\n===== \n";
447     os <<< "UQFF COMPARATIVE ANALYSIS – Research Student Module\n";
448     os <<< "t = <<< std::scientific <<< std::setprecision(4) <<<
449     os <<< "===== \n";

```

```

451
452 // Header row
453 os && std::left
454 && std::setw(4) && "Idx&quot;
455 && std::setw(44) && "System&quot;
456 && std::setw(15) && "g_base(m/s^2)&quot;
457 && std::setw(15) && "Ubi(m/s^2)&quot;
458 && std::setw(15) && "r(m)&quot;
459 && "M(kg)&quot; && "n&quot;;
460 os && std::string(110, &#39;-&#39;) && "n&quot;;
461
462 // Sort by g_base descending
463 std::array<int, NUM_SYSTEMS> order;
464 for (int i = 0; i < NUM_SYSTEMS; ++i) order[i] = i;
465 std::sort(order.begin(), order.end(),
466           [&](int a, int b){ return g_base_for(a) > g_base_for(b); });
467
468 for (int k = 0; k < NUM_SYSTEMS; ++k) {
469     int i = order[k];
470     double gb = g_base_for(i);
471     double ubi = compute_Ubi(i);
472     os && std::left
473 && std::setw(4) && i
474 && std::setw(44) && system_name(i)
475 && std::scientific && std::setprecision(4)
476 && std::setw(15) && gb
477 && std::setw(15) && ubi
478 && std::setw(15) && get_r(i)
479 && get_M(i) && "n&quot;;
480 }
481 os && std::string(110, &#39;-&#39;) && "n&quot;;
482 os && " Scale range (log10 r_max/r_min): && std::fixed && s
483 && compute_scale_range() && "decadesn&quot;;
484 os && " Advancement score: && std::fixed && std::setprecision(4)
485 && compute_advancement() && "%n&quot;;
486 os && "=====n&quot;
487 }
488
489 // Educational walkthrough: unique physics per system, dynamic catalogue, scale analysis
490 void printStudentGuide(std::ostream& os = std::cout) const {
491     os && "n=====n&quot;
492     os && " UQFF Research Student Guide – 7-System MUGE Seriesn&quot;;
493     os && " Universal Quantum Field Framework | CP3/PAPER_198 Buoyancy&quot;;
494     os && "=====n&quot;
495
496     os && "CORE ARCHITECTURE (applies to ALL systems)n&quot;;
497     os && "-----n&quot;;
498     os && " MUGE g(r,t) = sum of terms:n&quot;;
499     os && " term1 = g_base * (1+Hz*t) * (1-B/B_crit) * [unique corr per system]n&quot;;
500     os && " term2 = compute_Ug (Ug1+Ug2+Ug3+Ug4) * (1+f_TRZ)n&quot;;
501     os && " term3 = Lambda*c^2/3 (cosmological constant)n&quot;;
502     os && " term4 = EM correction with [UA] vacuumn&quot;;
503     os && " term_q, term_fluid, term_osc, term_DM, term_wind (shared)n&quot;;
504     os && " Buoyancy Pipeline (3 tiers, CP3/PAPER_198):n&quot;;
505     os && " Tier-1 term_Ubi = 0.5 * g_base [static half-gravity]n&quot;

```

```

506         os &&& &quot; Tier-2 term_F_UBi = -b_i * g * w_g * (M/r) * [UA] * cos(pi*t)\n&qu
507         os &&& &quot; Tier-3 term_Ub_i = -b_i * g * w_g * (M_ext/r_ext) * [UA] * cos(pi
508         os &&& std::scientific &&& std::setprecision(3);
509         os &&& &quot; b_i=&quot; &&& beta_i &&& &quot; w_g=&quot; &&& omega
510
511     for (int i = 0; i && NUM_SYSTEMS; ++i) {
512         os &&& &quot;System [&quot; &&& i &&& &quot;] &&& system_nam
513         os &&& &quot; Regime : &quot; &&& regime_label(i) &&& &quot;\n&quot;
514         os &&& std::scientific &&& std::setprecision(4);
515         os &&& &quot; M : &quot; &&& get_M(i) &&& &quot; kg (&quot;
516             &&& std::fixed &&& std::setprecision(0) &&& (get_M(i) / M_sun) &&&
517         os &&& std::scientific &&& std::setprecision(4);
518         os &&& &quot; r : &quot; &&& get_r(i) &&& &quot; m\n&quot;;
519         os &&& &quot; g_base : &quot; &&& g_base_for(i) &&& &quot; m/s^2\n&qu
520         os &&& &quot; Ubi : &quot; &&& compute_Ubi(i) &&& &quot; m/s^2\n&
521         os &&& &quot; Unique : &quot; &&& unique_physics_str(i) &&& &quot;\n
522     }
523
524     os &&& &quot;DYNAMIC PROCESSES CATALOGUE (7 unique time-dependent physics introduced)
525     os &&& &quot;-----\n&quot;;
526     os &&& &quot; 1. D(t) burst = D0*cos(omega_D*t)*exp(-t/tau_D) [Systems 0,1 – magn
527     os &&& &quot; 2. QPO burst = D0*cos(omega_D*t)*exp(-t/tau_D), omega_D=2pi/1200s [
528     os &&& &quot; 3. M(t) growth = M0*(1+Mdot*exp(-t/tau_SF)) [Systems 2,3,4,5 – a
529     os &&& &quot; 4. E(t) erosion = E0*exp(-t/tau_E) -&gt; corr_E=1-E(t)[System 5 – Pill
530     os &&& &quot; 5. H(z) Hubble = H0*sqrt(0m*(1+z)^3+OL) [System 6 – Rings,
531     os &&& &quot; 6. L_t lensing = (G*M/c^2*r)*L_fac -&gt; corr_L=1+L_t [System 6 – Ring
532     os &&& &quot; 7. Tidal force = 2*G*M_ext*r/r_ext^3 [Systems 0,1,2 – c
533
534     os &&& &quot;SCALE ANALYSIS\n&quot;;
535     os &&& &quot;-----\n&quot;;
536     os &&& std::scientific &&& std::setprecision(3);
537     os &&& &quot; r_min = &quot; &&& r_sgr1745 &&& &quot; m (magnetar, 20 km -
538     os &&& &quot; r_max = &quot; &&& r_rings &&& &quot; m (Einstein ring, 10
539     os &&& std::fixed &&& std::setprecision(2);
540     os &&& &quot; log10(r_max/r_min) = &quot; &&& compute_scale_range() &&& &qu
541
542     os &&& &quot;ADVANCEMENT METRICS\n&quot;;
543     os &&& &quot;-----\n&quot;;
544     os &&& std::fixed &&& std::setprecision(4);
545     os &&& &quot; diversity_score = &quot; &&& diversity_score
546         &&& &quot; (7/7 distinct physical regimes)\n&quot;;
547     os &&& &quot; dynamic_score = &quot; &&& dynamic_score
548         &&& &quot; (7/10 unique dynamic-physics processes)\n&quot;;
549     os &&& &quot; scalability_score = &quot; &&& scalability_score
550         &&& &quot; (16.2/20 log-decades of r coverage)\n&quot;;
551
552     os &&& &quot; coverage_score = &quot; &&& coverage_score
553         &&& &quot; (7/7 planned C++ modules complete)\n&quot;;
554     os &&& std::fixed &&& std::setprecision(1);
555     os &&& &quot; OVERALL ADVANCEMENT : &quot; &&& compute_advancement() &&& &qu
556     os &&& &quot;===== \n\n&quot;
557 }
558
559 // Compact parameter dump for all 7 systems
560 void printParameters(std::ostream& os = std::cout) const {
561     os &&& std::fixed &&& std::setprecision(3);

```

```

561         os &&& &quot;UQFFLearningAssessment (Research Student Edition)\n&quot;;
562         os &&& &quot; G=&quot; &&& G &&& &quot; c_light=&quot; &&& c_light &
563         os &&& &quot; beta_i=&quot; &&& beta_i &&& &quot; omega_g=&quot; &&&
564         for (int i = 0; i &lt; NUM_SYSTEMS; ++i) {
565             os &&& &quot; [&quot; &&& i &&& &quot;] &quot; &&& system_name(i)
566             &&& &quot; M=&quot; &&& std::scientific &&& get_M(i)
567             &&& &quot; r=&quot; &&& get_r(i)
568             &&& &quot; g_base=&quot; &&& g_base_for(i) &&& &quot;\n&quot;;
569         }
570         os &&& &quot; advancement=&quot; &&& std::fixed &&& std::setprecision(1) &
571     }
572
573     // Example: print guide + comparative analysis at t = 1 Myr
574     double exampleAt1Myr() const {
575         const double t = 1.0e6 * 3.156e7;
576         printStudentGuide();
577         printComparativeAnalysis(t);
578         return compute_advancement();
579     }
580
581     // ■■ setVariable / getVariable / addToVariable / subtractFromVariable ■■■■■■
582     bool setVariable(const std::string& varName, double newValue) {
583         if (varName == &quot;G&quot;;) G = newValue;
584         else if (varName == &quot;c_light&quot;;) c_light = newValue;
585         else if (varName == &quot;hbar&quot;;) hbar = newValue;
586         else if (varName == &quot;Lambda&quot;;) Lambda = newValue;
587         else if (varName == &quot;f_TRZ&quot;;) f_TRZ = newValue;
588         else if (varName == &quot;beta_i&quot;;) beta_i = newValue;
589         else if (varName == &quot;omega_g&quot;;) omega_g = newValue;
590         else if (varName == &quot;U_UA&quot;;) U_UA = newValue;
591         else if (varName == &quot;diversity_score&quot;;) diversity_score = newValue;
592         else if (varName == &quot;dynamic_score&quot;;) dynamic_score = newValue;
593         else if (varName == &quot;scalability_score&quot;;) scalability_score = newValue;
594         else if (varName == &quot;coverage_score&quot;;) coverage_score = newValue;
595         // SGR1745
596         else if (varName == &quot;M_sgr1745&quot;;) M_sgr1745 = newValue;
597         else if (varName == &quot;r_sgr1745&quot;;) r_sgr1745 = newValue;
598         else if (varName == &quot;B_sgr1745&quot;;) B_sgr1745 = newValue;
599         else if (varName == &quot;D0_sgr1745&quot;;) D0_sgr1745 = newValue;
600         else if (varName == &quot;M_ext_sgr1745&quot;;) M_ext_sgr1745 = newValue;
601         else if (varName == &quot;r_ext_sgr1745&quot;;) r_ext_sgr1745 = newValue;
602         // SGR0501
603         else if (varName == &quot;M_sgr0501&quot;;) M_sgr0501 = newValue;
604         else if (varName == &quot;r_sgr0501&quot;;) r_sgr0501 = newValue;
605         else if (varName == &quot;B_sgr0501&quot;;) B_sgr0501 = newValue;
606         else if (varName == &quot;D0_sgr0501&quot;;) D0_sgr0501 = newValue;
607         else if (varName == &quot;M_ext_sgr0501&quot;;) M_ext_sgr0501 = newValue;
608         else if (varName == &quot;r_ext_sgr0501&quot;;) r_ext_sgr0501 = newValue;
609         // SMBH Sgr A*
610         else if (varName == &quot;M_smbh&quot;;) M_smbh = newValue;
611         else if (varName == &quot;r_smbh&quot;;) r_smbh = newValue;
612         else if (varName == &quot;M_dot_smbh&quot;;) M_dot_smbh = newValue;
613         else if (varName == &quot;spin_smbh&quot;;) spin_smbh = newValue;
614         else if (varName == &quot;M_ext_smbh&quot;;) M_ext_smbh = newValue;
615         else if (varName == &quot;r_ext_smbh&quot;;) r_ext_smbh = newValue;

```

```

616         // Tapestry
617         else if (varName == &quot;M_tapestry&quot;;)      M_tapestry      = newValue;
618         else if (varName == &quot;r_tapestry&quot;;)      r_tapestry      = newValue;
619         else if (varName == &quot;rho_wind_tap&quot;;)    rho_wind_tap    = newValue;
620         else if (varName == &quot;v_wind_tap&quot;;)      v_wind_tap      = newValue;
621         else if (varName == &quot;M_ext_tap&quot;;)        M_ext_tap      = newValue;
622         else if (varName == &quot;r_ext_tap&quot;;)        r_ext_tap      = newValue;
623         // Westerlund 2
624         else if (varName == &quot;M_wd2&quot;;)           M_wd2          = newValue;
625         else if (varName == &quot;r_wd2&quot;;)           r_wd2          = newValue;
626         else if (varName == &quot;rho_wind_wd2&quot;;)    rho_wind_wd2    = newValue;
627         else if (varName == &quot;v_wind_wd2&quot;;)      v_wind_wd2      = newValue;
628         else if (varName == &quot;M_ext_wd2&quot;;)        M_ext_wd2      = newValue;
629         else if (varName == &quot;r_ext_wd2&quot;;)        r_ext_wd2      = newValue;
630         // Pillars
631         else if (varName == &quot;M_pillars&quot;;)        M_pillars      = newValue;
632         else if (varName == &quot;r_pillars&quot;;)        r_pillars      = newValue;
633         else if (varName == &quot;E_0_pillars&quot;;)      E_0_pillars    = newValue;
634         else if (varName == &quot;rho_wind_pil&quot;;)    rho_wind_pil    = newValue;
635         else if (varName == &quot;v_wind_pil&quot;;)      v_wind_pil      = newValue;
636         else if (varName == &quot;M_ext_pil&quot;;)        M_ext_pil      = newValue;
637         else if (varName == &quot;r_ext_pil&quot;;)        r_ext_pil      = newValue;
638         // Rings
639         else if (varName == &quot;M_rings&quot;;)          M_rings        = newValue;
640         else if (varName == &quot;r_rings&quot;;)          r_rings        = newValue;
641         else if (varName == &quot;Hz_rings&quot;;)         Hz_rings       = newValue;
642         else if (varName == &quot;L_factor_rings&quot;;)    L_factor_rings = newValue;
643         else if (varName == &quot;z_rings&quot;;)          z_rings        = newValue;
644         else if (varName == &quot;M_ext_rings&quot;;)      M_ext_rings    = newValue;
645         else if (varName == &quot;r_ext_rings&quot;;)      r_ext_rings    = newValue;
646         else {
647             std::cerr &lt;&lt; &quot;Error: Unknown variable &#39;&quot; &lt;&lt; varName &lt;&lt; &lt;&lt; endl;
648             return false;
649         }
650         if (logging_enabled)
651             std::cout &lt;&lt; &quot;[LOG] setVariable: &quot; &lt;&lt; varName &lt;&lt; &quot; = &lt;&lt; &lt;&lt; endl;
652         return true;
653     }
654
655     bool addToVariable(const std::string& varName, double delta) {
656         return setVariable(varName, getVariable(varName) + delta);
657     }
658
659     bool subtractFromVariable(const std::string& varName, double delta) {
660         return addToVariable(varName, -delta);
661     }
662
663     double getVariable(const std::string& varName) const {
664         if (varName == &quot;G&quot;;) return G;
665         else if (varName == &quot;c_light&quot;;) return c_light;
666         else if (varName == &quot;hbar&quot;;) return hbar;
667         else if (varName == &quot;Lambda&quot;;) return Lambda;
668         else if (varName == &quot;f_TRZ&quot;;) return f_TRZ;
669         else if (varName == &quot;beta_i&quot;;) return beta_i;
670         else if (varName == &quot;omega_g&quot;;) return omega_g;

```

```

671     else if (varName == "U_UA"){ return U_UA;
672     else if (varName == "diversity_score"){ return diversity_score;
673     else if (varName == "dynamic_score"){ return dynamic_score;
674     else if (varName == "scalability_score"){ return scalability_score;
675     else if (varName == "coverage_score"){ return coverage_score;
676     else if (varName == "M_sgr1745"){ return M_sgr1745;
677     else if (varName == "r_sgr1745"){ return r_sgr1745;
678     else if (varName == "B_sgr1745"){ return B_sgr1745;
679     else if (varName == "D0_sgr1745"){ return D0_sgr1745;
680     else if (varName == "M_ext_sgr1745"){ return M_ext_sgr1745;
681     else if (varName == "r_ext_sgr1745"){ return r_ext_sgr1745;
682     else if (varName == "M_sgr0501"){ return M_sgr0501;
683     else if (varName == "r_sgr0501"){ return r_sgr0501;
684     else if (varName == "B_sgr0501"){ return B_sgr0501;
685     else if (varName == "D0_sgr0501"){ return D0_sgr0501;
686     else if (varName == "M_ext_sgr0501"){ return M_ext_sgr0501;
687     else if (varName == "r_ext_sgr0501"){ return r_ext_sgr0501;
688     else if (varName == "M_smbh"){ return M_smbh;
689     else if (varName == "r_smbh"){ return r_smbh;
690     else if (varName == "M_dot_smbh"){ return M_dot_smbh;
691     else if (varName == "spin_smbh"){ return spin_smbh;
692     else if (varName == "M_ext_smbh"){ return M_ext_smbh;
693     else if (varName == "r_ext_smbh"){ return r_ext_smbh;
694     else if (varName == "M_tapestry"){ return M_tapestry;
695     else if (varName == "r_tapestry"){ return r_tapestry;
696     else if (varName == "rho_wind_tap"){ return rho_wind_tap;
697     else if (varName == "v_wind_tap"){ return v_wind_tap;
698     else if (varName == "M_ext_tap"){ return M_ext_tap;
699     else if (varName == "r_ext_tap"){ return r_ext_tap;
700     else if (varName == "M_wd2"){ return M_wd2;
701     else if (varName == "r_wd2"){ return r_wd2;
702     else if (varName == "rho_wind_wd2"){ return rho_wind_wd2;
703     else if (varName == "v_wind_wd2"){ return v_wind_wd2;
704     else if (varName == "M_ext_wd2"){ return M_ext_wd2;
705     else if (varName == "r_ext_wd2"){ return r_ext_wd2;
706     else if (varName == "M_pillars"){ return M_pillars;
707     else if (varName == "r_pillars"){ return r_pillars;
708     else if (varName == "E_0_pillars"){ return E_0_pillars;
709     else if (varName == "rho_wind_pil"){ return rho_wind_pil;
710     else if (varName == "v_wind_pil"){ return v_wind_pil;
711     else if (varName == "M_ext_pil"){ return M_ext_pil;
712     else if (varName == "r_ext_pil"){ return r_ext_pil;
713     else if (varName == "M_rings"){ return M_rings;
714     else if (varName == "r_rings"){ return r_rings;
715     else if (varName == "Hz_rings"){ return Hz_rings;
716
717     else if (varName == "L_factor_rings"){ return L_factor_rings;
718     else if (varName == "z_rings"){ return z_rings;
719     else if (varName == "M_ext_rings"){ return M_ext_rings;
720     else if (varName == "r_ext_rings"){ return r_ext_rings;
721     else {
722         std::cerr <<< "Error: Unknown variable &#39;" <<< varName <<<;
723         return 0.0;
724     }
725 }

```

[illegible]

```
781         double frac = (g_here != 0.0)
782             ? std::abs(g_other - g_here) / std::abs(g_here)
783             : std::numeric_limits<double>::quiet_NaN();
784         if (logging_enabled)
785             std::cout <<< " [XVAL] sys[" <<< sys_idx <<< " ] g_here=<math>g_{\text{here}}</math>
786                 <<< " g_other=<math>g_{\text{other}}</math>
787                 <<< " frac_diff=<math>g_{\text{other}} - g_{\text{here}}</math> <<< frac <<< std::endl;
788         return frac;
789     }
790 };
791
792 #endif // UQFF_LEARNING_ASSESSMENT_H
```

— End of UQFFLearningAssessment_001.cpp —